Contents

Executive Summary **4**

Project Decisions, Scope and Evidence Register **5**

1 Project Proposal **5**

1.1 Background and problem statement 5

1.2 Stakeholders and personas 6

1.3 Research aim, questions and hypotheses 6

1.4 Objectives and concrete deliverables 7

1.5 Expected outputs and acceptance philosophy 7

1.6 MVP boundary and exclusions 7

1.7 Research versus engineering contribution 8

1.8 Representative end-to-end decision scenario 8

2 Evidence of Relevance and Problem Validation **9**

2.1 Three distinct kinds of evidence 9

2.2 Exploratory review audit methodology 9

2.3 Stakeholder interview design 9

2.4 Validation hypotheses and minimum decision criteria 10

3 Literature Review **11**

3.1 Review scope and reproducible search strategy 11

3.2 Primary-source literature matrix 11

3.3 From document sentiment to aspect–polarity pairs 13

3.4 Language and domain mismatch 14

3.5 Method choice, complexity and interpretability 14

3.6 From model predictions to management analytics 15

3.7 Specific research gap and contribution claim 15

4 Comparison of Existing Systems **15**
4.1 Comparison framework 15
4.2 Functional comparison matrix 16
4.3 Differentiation without overclaiming 16
4.4 Suggested realistic competitive assessment 17

5 Technology Alternatives and Design Rationale **17**
5.1 Evaluation criteria 17
5.2 Frontend design decision 18
5.3 Backend and database decision 18
5.4 ML integration decision 19
5.5 Architecture decision record (ADR) template 19

6 Requirements Specification **19**
6.1 Use cases and prioritization 19
6.2 Functional requirements (FR) 19
6.3 Non-functional requirements (NFR) 22
6.4 Roles and authorization matrix 23
6.5 Business rules and precise indicator semantics 24
6.6 Traceability from requirement to artifact 25
6.7 Data rights and ethical constraints 25

7 Architecture, Data Model and System Diagrams **25**
7.1 Logical architecture overview 25
7.2 Diagram 1 — System context 26
7.3 Diagram 2 — Use cases and RBAC actors 27
7.4 Diagram 3 — Component architecture 27
7.5 Diagram 4 — Preliminary entity–relationship model 28
7.6 Diagram 5 — Import and inference sequence 29
7.7 Core entities and integrity constraints 29
7.8 API contract sketches 30
7.9 Import and inference failure handling 31
7.10 Architecture verification checklist 32

8 UX, Information Architecture and Prototype **32**
8.1 UX objectives and content hierarchy 32
8.2 Eight map-first prototype screens 32
8.3 Interaction scenario U1 — Owner investigates speed complaints . 33
8.4 Interaction scenario U2 — Manager validates an aspect label . . 34
8.5 Interaction scenario U3 — Administrator imports CSV safely . . 34
8.6 Prototype availability and fidelity 34
8.7 Accessibility, responsiveness and honest analytics display 34
8.8 Proposed usability tasks and screen acceptance 35

9 Research Methodology, Verification and Evaluation Plan **35**
9.1 Evaluation design: four independent questions 35
9.2 Corpus provenance, data selection and split discipline 35
9.3 Annotation protocol and ambiguity adjudication 36
9.4 Model experiment design 36
9.5 Model metrics and reporting templates 37
9.6 Error analysis and domain robustness 38
9.7 Software, security and reliability tests 38
9.8 User study and managerial task comparison 39
9.9 Reproducible evidence record 39

10 Project Management, Collaboration and Risks **40**
10.1 Proposed responsibilities 40
10.2 Source control and issue management 40
10.3 Risk register 40
10.4 Definition of done 41

11 Academic Bibliography **42**

12 Official Product and Technology Source Register **44**

A Proposed Minimal CSV Schema **44**

B Sample Annotation Cases for Reviewer Discussion **45**

Diploma topic: Development of a Web-Based User Review Analysis System to Support Managerial Decision-Making in Food Service Chains.

Working product name: ReviewLens (provisional; subject to change during development).

Team members: Zhambulkhan Dinmukhamed; Aitkazyyev Serik.

Academic supervisor: Orazova Arailym Zhanbolatkyzy.

Executive Summary

ReviewLens is a proposed decision-support web application for food-service chains in Kazakhstan. It is designed to process Russian-language, Kazakh-language and mixed Russian–Kazakh customer reviews, including two languages appearing within one review. Reviews will be acquired through permitted channels. A machine-learning pipeline will identify six operational aspects—food, service, price, atmosphere, cleanliness, and speed—and assign a positive, neutral, or negative sentiment to each aspect actually mentioned. For example, one review may praise food while criticizing waiting time, even when its wording switches between Russian and Kazakh. The intended users are chain owners, branch managers, and system administrators. Managers will examine branch-level trends and inspect original reviews through a web interface before deciding whether an issue warrants investigation.

The working minimum viable product (MVP) combines a React/TypeScript interface, a Spring Boot REST backend, PostgreSQL storage, a Python/FastAPI NLP service, role-based access, CSV ingestion, bilingual and mixed-language review processing, review filtering, branch comparison, and traceable analytics. After sign-in, an authorized user will see an Astana-focused branch map and a side navigation menu; selecting a branch will open its analytics. External-platform connectors remain conditional on authorized access. Support for all three language groups is a project requirement.

The intended research contribution is a **reproducible evaluation of aspect-based sentiment analysis (ABSA) for Russian, Kazakh, and mixed Russian–Kazakh food-service reviews**, together with a study of whether aspect-level dashboards help managers find recurring branch problems more accurately and efficiently than a conventional review list. The ML component is expected to detect mentioned aspects and classify the sentiment expressed toward each one; results will be evaluated separately for the three language groups. The engineering contribution is a web-based workflow that imports reviews from permitted sources, stores the original text and versioned model outputs, and displays review counts, aspect trends, and supporting examples. Access will be restricted by organization and branch: owners can see their own network, while managers can see only the branches assigned to them.

Project Decisions, Scope and Evidence Register

Design issue	Working decision	Evidence/status and action
Comparison	Branches of the same organization, not competitors	Proposed MVP boundary
Access model	Owner, assigned branch manager, administrator	Proposed; server-side enforcement required
Application stack	React + TypeScript; Spring Boot; PostgreSQL; Python FastAPI	Architecture proposal, not integrated implementation
Evaluation	Multilingual ABSA baseline versus multilingual encoder; per-language-group results; usability comparison	Planned; no experimental results provided
Interactive UX	Six-screen, browser-accessible demonstration with synthetic data	Prototype exists; frontend demo does not prove backend/ML integration

1 Project Proposal

1.1 Background and problem statement

Food-service customers publish star ratings and narrative reviews on mapping, discovery, and recommendation platforms. A scalar rating compresses heterogeneous experiences: good food can coexist with poor cleanliness, high prices, or long waiting times. At chain level, differences between branches may be masked by aggregated averages. Staff may manually read recent reviews, but manual analysis becomes harder as the number of branches, sources, languages, and reviews increases. Relevant details can be overlooked, while categorical conclusions can be based on a few salient complaints without an explicit denominator.

Commercial reputation-management platforms already provide review aggregation and certain NLP-enabled analytics. Accordingly, ReviewLens does **not** claim to invent reputation monitoring. Its narrower motivation is to test a transparent, locally relevant combination of multilingual Russian–Kazakh ABSA (including code-switching within reviews), six food-service operations categories, branch-level comparisons, and the ability to trace numerical indicators to the reviews that support them. A prospective user may care less about an additional chart than about whether a chart has correct labels, sufficient observations, and a clear path to an actionable source review.

1.2 Stakeholders and personas

Chain owner — strategic overview. Responsible for several branches and interested in trends, cross-branch variation, sample sizes, and operational priorities. The owner needs an organization-level dashboard and the ability to open a negative-aspect cluster to inspect the supporting comments.

Branch manager — operational diagnosis. Oversees one or a restricted set of branches. The manager needs date/source/aspect filters and annotated original reviews; must never gain access to unassigned branches by modifying a URL or request parameter.

System administrator — data and account stewardship. Manages users, branch assignments, import permissions, and ingestion health. An administrator's technical role does not automatically grant unrestricted access to every organization's analytic data.

Secondary stakeholders include the academic supervisor, anonymized research participants, customers whose reviews may be processed, and the team member responsible for data protection. This is a *decision-support system*, not a consumer-facing review publication platform.

1.3 Research aim, questions and hypotheses

Overall aim. Design, implement, and evaluate a web application that transforms lawfully accessible food-service reviews into aspect-level sentiment indicators and comparative branch analytics, supporting managers' informed investigation of recurring problems.

RQ1 — multilingual model performance. How accurately can a statistical baseline and a fine-tuned multilingual encoder identify six predefined aspects and their associated sentiments in Russian-only, Kazakh-only and code-switched Russian–Kazakh food-service reviews, and how does performance differ between these language groups?

RQ2 — managerial task effectiveness. In a controlled pilot, does an aspect-level dashboard enable users to identify recurring branch issues more quickly or accurately than a conventional review list with overall ratings?

RQ3 — integration feasibility. Can permitted review ingestion, model inference, role-restricted access, and consistent branch-level aggregation be integrated into an auditable MVP within the project's engineering constraints?

H1. A multilingual transformer fine-tuned on appropriately annotated target-language data achieves higher end-to-end aspect–polarity F1 than the statistical baseline on an independent test set. Its results will also be reported for Russian-only, Kazakh-only and mixed-language subsets; improvement in each subset is not assumed. **H2.** A dashboard that combines aspect-level sentiment, review

counts, and drill-down examples improves task completion over a conventional review-list interface in a small usability comparison. These are falsifiable hypotheses, not existing findings. **H3.** A CSV-first architecture remains operational when third-party API permissions are unavailable; this is an engineering acceptance hypothesis.

1.4 Objectives and concrete deliverables

1. Elicit the most useful review-analysis scenarios through documented stakeholder conversations and a controlled pilot inspection of accessible reviews.
2. Define stable labels for six aspects and three polarity classes, including multi-aspect, mixed-polarity, Kazakh-only, Russian-only and code-switched review examples.
3. Construct a permitted, de-identified and multilingual experimental corpus with documented source, per-language sample counts, inclusion criteria, splits, and bilingual annotation agreement.
4. Establish a reproducible word/character n-gram statistical baseline, then compare at least one multilingual transformer with the same held-out data across RU, KK and mixed RU–KK subsets.
5. Build an organization-aware database and secure REST API, with CSV import and asynchronous job status.
6. Integrate inference results without granting the ML service unrestricted database access.
7. Implement the map-first workspace for Astana branches, analytics dashboard, reviews, imports, exports and access settings, with drill-down and relevant empty/error states.
8. Evaluate functionality, security, NLP accuracy, latency, and human task performance against pre-specified tests.
9. Prepare diagrams, source-controlled code, architecture decisions, experimental artifacts, and a limitations statement for the final defense.

1.5 Expected outputs and acceptance philosophy

The expected result is a demonstrable end-to-end prototype, reusable source code and data contracts, a traceable corpus and annotation protocol, experiment tables (including unsuccessful methods), diagrams, and an academic report. ReviewLens is useful only if it produces indicators with an intelligible denominator, does not silently mix model versions, and respects branch-level permissions.

1.6 MVP boundary and exclusions

Must-have MVP: one logical organization with multiple branches and a schema capable of supporting more; Russian, Kazakh and mixed Russian–

Kazakh reviews; CSV import; six fixed aspects; three sentiment labels per detected aspect; organization/branch permission checks; import status; a map-based Astana branch home; branch and chain dashboards; detailed review filters; and same-organization branch comparisons. The language identifier must distinguish `ru`, `kk`, `mixed_ru_kk` and unsupported/uncertain cases without treating a Kazakh review as Russian merely because it includes Cyrillic text.

Should-have: overall review sentiment as a separate task, downloadable CSV reports, accessibility refinements, and structured usability evaluation. **Conditional extensions:** Google Business Profile connector with authorized access, possible 2GIS connector if an approved interface is confirmed, XLSX export, email/Telegram notifications. **Explicitly out of MVP:** competitor scraping, automated replies on behalf of businesses, prescriptive automatic managerial decisions, BERTopic discovery, and LLM-generated recommendations. Kazakh and mixed Russian–Kazakh analysis are **in** scope; convincing quality claims are permitted only after group-specific evaluation. Proposed extensions must not appear as implemented functionality in diagrams, screenshots, or evaluation tables.

1.7 Research versus engineering contribution

The scientific part tests classification and usability hypotheses under controlled evaluation. The engineering part provides an integrated, security-aware application. These outputs reinforce one another but have different evidence standards: a functioning frontend proves screen behavior, not classification quality; an F1 score proves performance on a specific labeled corpus, not commercial return on investment. Causal claims about sales, turnover, or operational improvement would require a separate business study.

1.8 Representative end-to-end decision scenario

A chain owner selects the same four-week period for Branch A and Branch B and notices a greater negative share for *speed* in Branch A. The dashboard displays the number of reviews mentioning speed in each branch; the owner filters to negative speed comments, reads their original context, and records a possible investigation into kitchen capacity or pickup congestion. The dashboard does not assert causality. If the number of relevant reviews is too small or a data source has failed to update, the interface surfaces that limitation before the user draws a conclusion.

2 Evidence of Relevance and Problem Validation

2.1 Three distinct kinds of evidence

Scientific relevance is supported by prior ABSA work showing why one text may require multiple aspect labels [S1, S2]. **Product landscape evidence** shows that review management is already a meaningful software category, including multi-location solutions [W1–W5]. **Local customer relevance**—whether managers in Kazakhstan face this exact pain point and would benefit from these displays—must be established through primary interviews and observation. The first two do not substitute for the third.

2.2 Exploratory review audit methodology

The proposed exploratory review audit methodology targets approximately **300–500 lawfully accessible text reviews from 5–10 branches or food-service businesses in Kazakhstan**, subject to permissions and available data. The selection protocol records the time period, city, business type, sampling procedure, maximum items per branch, and exclusions. Sampling exclusively one-star reviews is intentionally avoided to prevent distorting prevalence estimates.

For each admissible record, the research register contains a pseudonymous review identifier, platform/source, branch, publication date, available rating, detected language (`ru`, `kk`, mixed, or other), availability of meaningful text, candidate aspects and polarities, reply flag and dates when available, and reason for exclusion if relevant. The analytical copy strictly excludes usernames and unnecessary personally identifiable material. A public-facing dashboard does not expose confidential research exports.

Exploratory outputs systematically evaluate: language distribution; six-aspect frequency; co-occurrence; negative-polarity share per aspect; number of star-only entries; ratio of replied-to negative reviews where response metadata actually exists; and variance between branches over a consistent period. Each analytical table includes **n**, a strict denominator definition, collection dates, and handles percentages with missing data explicitly.

2.3 Stakeholder interview design

Target sample: 3–5 owners, branch managers, or relevant staff members, engaged through a 15–20 minute semi-structured interview with voluntary consent. The research protocol requires explaining academic purposes, avoiding

unnecessary audio recording, and giving participants an option to omit sensitive company information. Interview notes are anonymized using codes such as P01 rather than personal names.

Interview guide outline:

1. Which platforms currently receive your branch reviews? Who reads them, and how often?
2. What is the approximate time spent monitoring reviews each week? Can it be observed rather than estimated?
3. How are recurring service, speed, cleanliness, or food complaints identified?
4. Do you compare branches; if so, which indicators are used and what is difficult?
5. What do present platforms do well, and which unmet need is most important?
6. Have highly rated reviews contained serious complaints or vice versa?
7. What would you do after observing a rise in negative *speed* comments at one branch?
8. How would you prefer alerts, filters, exports, and source links to behave?
9. How much uncertainty or model error is acceptable before you trust a chart?
10. Which languages occur in reviews? What restrictions exist on storage and redistribution?

The research design specifies that themes will be identified through structured notes and a secondary review of coding decisions. Interview recruitment and qualitative analysis are conducted independently from the academic literature review to ensure unbiased elicitation of local operational requirements.

2.4 Validation hypotheses and minimum decision criteria

Hypothesis	Planned observation	What would challenge the assumption
Branch comparison is valuable	At least some participants perform cross-branch diagnosis or request it	All participants need only single-branch monitoring
Aspect labels are intelligible	Users can correctly interpret food/service/speed in a short task	Repeated confusion between the selected categories
Drill-down improves trust	Users request supporting original comments before acting	Users disregard source review/context even when available
CSV is a workable entry point	An authorized partner can export/prepare required fields	Available data are inaccessible or unlicensed for the task

Hypothesis	Planned observation	What would challenge the assumption
Multilingual MVP is feasible	Permitted reviews and annotators cover RU, KK and mixed RU–KK; held-out samples allow each group to be examined	Too few Kazakh/mixed examples or poor measured performance would require a documented scope/quality revision

3 Literature Review

3.1 Review scope and reproducible search strategy

The review addresses ABSA problem definitions; Russian and Kazakh sentiment resources; baseline versus contextual encoders; aspect-oriented review dashboards; and evaluation gaps involving data/domain transfer. It prioritizes publisher papers, ACL Anthology, verified arXiv versions, and model cards rather than treating informal summaries as scientific evidence. Records should be deduplicated by DOI, publisher URL, and work title. The selected foundational works may predate 2021; newer works should be screened through 2026 using actual database timestamps.

The following Scopus **Advanced Search strings formulate the core retrieval strategy**:

```
TITLE-ABS-KEY(("aspect-based sentiment analysis" OR "aspect category
↪  sentiment"
 OR "aspect-level sentiment" OR "opinion mining")
 AND (restaurant* OR cafe* OR "food service" OR hospitality OR "online
 ↪  review*"))
AND PUBYEAR > 2020 AND PUBYEAR < 2027
```

```
TITLE-ABS-KEY(("sentiment analysis" OR "aspect-based sentiment")
 AND (Russian OR Kazakh OR multilingual OR "low resource")
 AND (review* OR restaurant* OR hospitality))
AND PUBYEAR > 2020 AND PUBYEAR < 2027
```

Inclusion criteria: clear relation to the task, dataset, language, method, or product evaluation; accessible abstract or full text; bibliographic identity verified at a primary source; identifiable experimental context where accuracy claims are considered. Exclusion criteria: duplicate versions counted twice; unsupported marketing claims presented as science; papers whose sentiment target is unrelated; untraceable citation metadata.

3.2 Primary-source literature matrix

ID	Publication and contribution	Limitation for ReviewLens	Design implication
S1	Pontiki et al. (2016), SemEval-2016 Task 5: formal aspect-category and sentiment tasks, including restaurant data	Benchmark/task conventions are not identical to local Russian–Kazakh branch reviews	Define exact label schema before benchmarking
S2	Hua, Denny, Wicker & Taskova (2024), systematic ABSA review: **727 primary studies screened from 8,550 search results**; reports limited dataset/domain diversity and domain mismatch	Review-level tendencies do not establish local model performance	Report target-domain generalization and source diversity
S3	Rogers et al. (2018), RuSentiment: **31,185** annotated Russian VK posts; Fleiss' κ **0.58** under a three-labeler procedure	Social media sentiment differs from restaurant aspects	Reuse lessons for annotation rigor, not as an ABSA gold test
S4	Kotelnikova et al. (2022), lexicon methods versus BERT for text sentiment analysis	Different datasets and tasks than the six-aspect proposal	Include an interpretable baseline and equal test conditions
S5	Smetanin & Komarov (2019), Russian product-review sentiment with CNNs	Product ratings do not directly annotate individual aspects	Avoid substituting star rating for aspect ground truth
S6	Zmitrovich et al. (2024), pretrained Russian transformer family	Pretraining alone is not food-service validation	Evaluate fine-tuning and compute requirements
S7	`cointegrated/rubert-tiny2`, public model card	Russian-only pretraining is not evidence for robust KK or mixed-language analysis	Optional Russian-only diagnostic, not the primary multilingual candidate
S8	Yeshpanov & Varol (2024), KazSAnDRA Kazakh sentiment resource	Overall sentiment resource does not supply our six aspect labels	Create and test Kazakh aspect-specific gold labels

ID	Publication and contribution	Limitation for ReviewLens	Design implication
S9	Grootendorst (2022), BERTopic topic-modeling approach	Topics are data-driven and may not match manager-relevant categories	Optional discovery, not an MVP replacement for ABSA
S10	Jayakody, Thenahandi & Jayarathna (2026), SentimentLens hospitality preprint	Different venue domain; preprint status	Inspiration for multilevel aggregation, not proof of transfer
S11	Scaria et al. (2024), InstructABSA with instruction-tuned ABSA tasks	Published evaluation does not establish Russian/KK transfer	Candidate future method only after comparable tests
S12	Fehle et al. (2023), multilabel ABSA in German hotel reviews	Different language and business domain	Evidence that multilabel framing merits testing
S13	Rybakov & Malafeev (2018), Russian hotel ABSA	Hospitality categories and available corpus differ	Do not claim Russian-domain ABSA is unprecedented
S14	Loukachevitch et al. (2015), SentiRuEval Russian targeted sentiment	Historical task/task distribution differs from product labels	Consult Russian benchmark conventions and history
S15	Conneau et al. (2020), XLM-R multilingual representation learning [ACL Anthology]	General multilingual pretraining is not validated Kazakh–Russian food-service ABSA	Evaluate a fine-tuned multilingual candidate with RU/KK/mixed test strata

3.3 From document sentiment to aspect–polarity pairs

A single sentiment label can be misleading when contradictory opinions occur in one review. Example: *"The soup was great, but the queue took forty minutes."* Document polarity is not enough to identify which operational process is associated with a complaint. The proposed annotation stores a set of present aspects and their own polarity labels, such as `food: positive` and `speed: negative`. This is a multilabel aspect-detection task followed by conditional aspect sentiment classification. "Not mentioned" must be distinguished from the expressed sentiment "neutral"; otherwise denominator counts and negative percentages become incorrect.

For every review and candidate aspect, the model should estimate mention presence; polarity is meaningful only for the detected or gold-mentioned aspects, depending on the experimental stage. To avoid optimistic scores, the final end-to-end metric must evaluate whether the correct *pair* was returned, not merely

whether polarity was correct when the true aspect was supplied. The study must report both the component-level and pipeline-level outcomes.

3.4 Language and domain mismatch

The published systematic review [S2] documents a shortage of diversity in the datasets and domains used in ABSA research. This is particularly relevant here: well-known English-language restaurant corpora, social-media Russian sentiment, Russian hotels, and actual food-service reviews in Kazakhstan represent related but non-identical distributions. Colloquialisms, local dish names, spelling mistakes, platform conventions, and code-switching are plausible sources of error. The project must **test** these risks, not imply the model already understands them.

Russian-language research is not absent. RuSentiment [S3] is a substantial annotated sentiment resource but not a six-aspect restaurant dataset. Historical Russian targeted sentiment tasks [S13–S14] also rule out claiming global novelty for Russian ABSA. Kazakh sentiment resources [S8] establish adjacent work, but their general polarity labels are not ground truth for food, price or speed aspects. Multilingual pretraining such as XLM-R [S15] creates a plausible method candidate, **not** proof that mixed-language reviews can be classified correctly. The defensible gap is an evaluated **Kazakhstan food-service branch workflow and six-aspect taxonomy** with Russian-only, Kazakh-only and code-switched test cases.

3.5 Method choice, complexity and interpretability

A word/character TF-IDF linear model supplies a transparent multilingual-text benchmark; its difficulty with implicit sentiment and context should be measured rather than presumed. Transformer encoders can capture context and may improve classification, but require suitable labels and may be harder to deploy. A multilingual candidate such as XLM-R [S15] is therefore more appropriate as the principal cross-language experiment than an exclusively Russian encoder. Russian-specific models can be tested on the RU subset as diagnostic comparators, but cannot establish Kazakh or mixed-language performance. All methods must use controlled test splits, preprocessing and threshold selection; inference speed and memory footprint count as outcomes alongside F1.

Instruction-based approaches [S11] are promising in the literature, but reported English SemEval results cannot be presented as expected results in the proposed corpus. Topic modeling [S9] can reveal unsupervised clusters, yet such clusters may be unrelated to a manager's operating categories. Both remain extensions until the baseline and six-aspect contract are validated.

3.6 From model predictions to management analytics

Aggregated visualizations should allow users to see branch, period, aspect, proportion, and evidence together. Related hospitality-analysis work [S10] illustrates that such systems are not a new genre. ReviewLens's proposed academic value is in transparent input/output definitions and two evaluations: whether the NLP model assigns reliable labels to target-domain reviews, and whether the resulting interface helps users locate relevant observations. A correlation between complaints and poor ratings is not proof of a causal relationship with sales or staffing.

3.7 Specific research gap and contribution claim

Gap statement: Within the reviewed literature, a need remains to evaluate the transfer of multilingual ABSA into a Kazakhstan food-service workflow with **Russian, Kazakh and code-switched reviews**, multiple aspect–polarity outputs, traceable original evidence and branch comparison restricted to authorized organizations. This is a scoped project gap, **not** a claim that no earlier product or research performed related analysis. The final literature review must revise this claim if the completed Scopus search finds directly comparable systems or datasets.

Proposed contribution: (i) an explicit six-aspect annotation scheme and adjudicated target-domain evaluation set spanning three language groups; (ii) a baseline-versus-multilingual-transformer comparison with per-language and per-aspect errors and resource cost; (iii) an integrated map-first multi-branch analytics prototype; and (iv) a small controlled task study addressing usefulness. Whether these contributions are achieved is determined by evidence in later stages.

4 Comparison of Existing Systems

4.1 Comparison framework

This section evaluates representative **commercial and platform-native tools** against the proposed academic system, using functionality rather than an unsupported claim of market uniqueness. The comparison dimensions are: platform coverage; review-response workflow; sentiment analysis; aspect/category detail; multi-location analytics; intra-chain comparison; evidence for Russian/Kazakh task quality; export/integration; alerts; and Kazakhstan availability. Product prices, algorithmic accuracy, and current access privileges are not inferred when official evidence is missing. A statement marked *unconfirmed* is not equivalent to "the feature does not exist."

4.2 Functional comparison matrix

Product	Documented or described capability	Not established for the local research case	Implication for ReviewLens
Google Business Profile [W1]	Review management for verified/managed business profiles; responses and multi-location account management; authorized API workflows	Six-label RU/KK ABSA quality, access to unrelated business data	Candidate adapter for consenting branch owners only
2GIS for business [W2]	Native business review/rating workflow within the 2GIS ecosystem	Permission and public endpoint for bulk external extraction; aspect-polarity coverage	Do not promise a scraper; preserve CSV alternative
Yandex Business [W3]	Organization review management and response/notification workflow	External export/ABSA features and exact Kazakhstan availability under proposed terms	Useful platform-native baseline for manager interview
ReviewTrackers [W4]	Vendor-described NLP experience categories, sentiment and location-performance analysis	Performance specifically on Kazakhstan Russian reviews, price, permitted local sources	Real competitor in category analytics, not an absence-of-features straw man
Birdeye [W5]	Vendor-described review consolidation, sentiment insights and multi-location reports	Six-category RU/KK evaluation, local data connectors and commercial terms	Real alternative for multi-location reporting; research must focus on transparent local validation
ReviewLens (proposed)	Planned CSV ingestion, six aspect–polarity categories, own-branch comparison, RBAC, evidence-linked analytics	All actual model scores, deployed integration, user adoption	Academic transparency, reproducibility and scoped operational vocabulary

4.3 Differentiation without overclaiming

ReviewLens aims to stand out on **traceability and target-domain evaluation**: each negative-aspect indicator should resolve to permitted source text and a documented model version; counts should be computed consistently for two branches; and the report should quantify model performance specifically on

the chosen corpus. Existing products already offer mature workflows, so developing a new application is an engineering/research decision rather than proof that comparable software is commercially absent.

4.4 Suggested realistic competitive assessment

During stakeholder interviews, show a one-page matrix of the participants' currently used platform and the ReviewLens mockup. Ask which concrete task is easier in each, and whether a dedicated aspect dashboard would save time. Record features as *verified by official documentation*, *observed by participant*, *not tested*, or *claimed by vendor*.

5 Technology Alternatives and Design Rationale

5.1 Evaluation criteria

The team should compare tools using development familiarity, completeness of RBAC support, relational consistency, deployability, licensing, inference costs, testability, interoperability, security, and whether a simpler architecture can satisfy the MVP. The proposal favors reliable integration over maximizing framework count. Hardware budgets, GPU availability, and operational costs must be filled in from actual project resources.

Layer / decision	Alternatives	Selected working option	Justification and unresolved tradeoff
API backend	Spring Boot; NestJS; Django	Java Spring Boot	Spring Security, transactions and integration tests; requires JVM deployment
Persistence	PostgreSQL; MySQL; MongoDB	PostgreSQL	Joins and integrity for organizations, branches, reviews and labels; JSONB only for optional source metadata
Inter-service execution	Synchronous REST; DB-backed jobs; message broker; gRPC	Public REST plus background work queue/job table	Lower MVP operational burden; message broker only if real workload demands it

Layer / decision	Alternatives	Selected working option	Justification and unresolved tradeoff
Text classification	Word/character TF-IDF; multilingual XLM-R; multilingual BERT-family alternatives; Russian encoder as RU diagnostic	TF-IDF baseline and multilingual XLM-R candidate	Compare RU/KK/mixed aspect–polarity F1 and latency; XLM-R is not guaranteed accurate before fine-tuning
ML endpoint	Python FastAPI; Java native inference; hosted inference	Python FastAPI, internal only	Practical Python NLP tooling and model versioning; adds service boundary
Ingestion	CSV; authorized business APIs; browser scraping	CSV Must; official API Could	Legal clarity and resilient offline demonstration; no unauthorized scraper
Deployment	Docker Compose; Kubernetes; platform services	Container-based single-server or demo environment, to decide	Compose likely sufficient for an academic prototype; deployment evidence pending

5.2 Frontend design decision

The UI will be based on a common shell with navigation, date/branch selectors, permission-sensitive routes, and reusable chart/list components. TypeScript interfaces should be generated from OpenAPI where feasible, preventing a chart from expecting fields that the backend does not return. A cache or React Query-style data-fetching library may simplify loading and failure states but must not become the only enforcement point for access control.

5.3 Backend and database decision

Spring Boot owns all externally accessible routes, authentication, authorization, business validation, transactions, persistence, and analytics query semantics. The database should enforce foreign keys, useful indexes on branch/date/aspect and uniqueness for deduplication. Schema changes must be version-controlled, for example with Flyway. A business organization means a restaurant chain account, not a rented property or a commercial tenant. The backend determines which organization and branches a signed-in user may access; it never trusts a branch ID supplied by the browser without checking it.

5.4 ML integration decision

Python handles preprocessing and inference; Spring Boot remains responsible for saving predictions to PostgreSQL. The ML service responds with a model identifier, class list, optional confidence, and errors. A change in model version should create a new `Analysis` record, not quietly overwrite all historical interpretation. In the simplest architecture, background work is polled from a durable job table; avoid Kafka/RabbitMQ unless throughput or team constraints justify the maintenance overhead.

5.5 Architecture decision record (ADR) template

Every contested design choice should be documented as: **Context → Options → Decision → Consequences → Revisit trigger**. Immediate ADR candidates are `ADR-001: CSV-first import`, `ADR-002: fixed six-aspect taxonomy`, `ADR-003: Spring Boot + internal Python NLP`, and `ADR-004: isolation of branches and organizations`.

6 Requirements Specification

6.1 Use cases and prioritization

The MoSCoW categories are **Must**, **Should**, **Could**, and **Won't in MVP**. Acceptance criteria define how later implementation will be tested; they are not test results. Every Must feature should map to a documented route, role, screen, and verification case. The review taxonomy must remain identical in database constraints, ML model labels, API DTOs, charts, and filters.

6.2 Functional requirements (FR)

ID	Priority	Requirement	Observable acceptance condition
FR-01	Must	Authenticate registered users	Valid credentials create an authenticated session/token; invalid credentials fail; anonymous protected requests are rejected

ID	Priority	Requirement	Observable acceptance condition
FR-02	Must	Enforce organization and branch RBAC	Owner sees own restaurant chain; manager only assigned branches; attempts to retrieve another chain's or unassigned branch's data are denied
FR-03	Must	Manage chain branches and map locations	Authorized owner/admin maintains branch name, address, coordinates when available, and archive status
FR-04	Must	Import CSV reviews	Valid rows create reviews; malformed rows produce specific errors; reimport creates no duplicates
FR-05	Must	Track asynchronous processing	Job reports queued, running, completed, completed_with_errors, or failed, with accepted/rejected counts
FR-06	Must	Preserve review text, source and language	Store original RU, KK or mixed RU–KK text, detected language label, date, branch, source, ID/dedup key, and optional rating
FR-07	Must	Identify six predefined aspects	One review may have multiple detected codes; outputs use the six stable values only
FR-08	Must	Classify sentiment per present aspect	Each predicted present aspect has positive/neutral/negative; absent aspect is not relabeled neutral
FR-09	Should	Compute optional overall sentiment	Independent three-class field; does not replace aspect-specific outputs

ID	Priority	Requirement	Observable acceptance condition
FR-10	Must	Present branch and chain dashboard	Period, sample counts, aspect prevalence and negative share are recomputable from database records
FR-11	Must	Filter and inspect reviews	Language (RU/KK/mixed), source, date, branch, aspect and polarity filters work together; original review context remains visible
FR-12	Must	Compare own branches	Authorized owner compares at least two branches over the same period with normalized and absolute counts
FR-13	Should	Export permitted data to CSV	Export matches current server-validated filters and includes period and UTF-8 encoding
FR-14	Could	Connect permitted Google Business Profile data	Only for authenticated/verified managed locations with authorized credentials; CSV still works without connector
FR-15	Could	Connect to 2GIS if permitted	Implement only with documented legitimate access and usage restrictions; otherwise mark deferred
FR-16	Could	Provide configurable notifications	One unique negative event generates at most one notification; duplicates suppressed
FR-17	Could	Export XLSX	File opens reliably and preserves applied filters/date metadata

ID	Priority	Requirement	Observable acceptance condition
FR-18	Must	Analyze Russian, Kazakh and code-switched reviews	Supports RU-only, KK-only and RU–KK mixed text, including switching within a sentence; per-group performance is evaluated, not presumed
FR-19	Won't	Monitor and compare competitors	Must not appear as a required supported workflow in MVP UI or UML
FR-20	Won't	Auto-generate business decisions	Human users decide; system shows evidence and uncertainty instead
FR-21	Must	Show an interactive Astana branch map	After sign-in, map pins show only authorized branches; selecting a pin opens branch analytics; list fallback supports missing coordinates

6.3 Non-functional requirements (NFR)

ID	Priority	Quality attribute	Planned measurement and criterion
NFR-01	Must	Aspect detection quality	Per-label F1 plus Micro/Macro-F1 on held-out gold set, separately for RU, KK and mixed RU–KK; provisional Micro-F1 target $\geq$0.70, not measured
NFR-03	Should	Overall sentiment quality (if FR-09)	Three-class Macro-F1; provisional target $\geq$0.80, not measured
NFR-04	Must	Analytics API latency	Fixed dataset of at least 10,000 reviews, 50 simulated users, p95 GET $\leq$2 seconds; report failures and hardware; target only

ID	Priority	Quality attribute	Planned measurement and criterion
NFR-05	Must	Import reliability	Per-row errors isolated, job resumes safely, source IDs/hash rules prevent duplicate materialization
NFR-06	Must	Security and privacy	Strong password hashing, TLS in deployment, backend checks for organization and branch access, secrets omitted from logs/Git
NFR-07	Must	Reproducibility and observability	Job/model version, timestamps, data hashes, training configuration and diagnostic status persisted without personal-data spill
NFR-08	Should	Usability	Target SUS $\geq$70 plus task time/error/success measures from actual participants; not measured
NFR-09	Should	Source extensibility	Contract test demonstrates a new synthetic adapter without restructuring core analytics schema
NFR-10	Should	Basic accessibility	Keyboard-operable principal flows, labeled forms, sufficient contrast and text alternative to charts

6.4 Roles and authorization matrix

Activity	Chain owner	Branch manager	Administrator
View all own-chain analytics	Yes	No	No by default
View assigned-branch analytics	Yes, own chain	Yes, assigned only	Only when explicitly authorized
Read permitted original reviews	All own branches	Assigned branch	Diagnostic access only if delegated

Activity	Chain owner	Branch manager	Administrator
Add / archive branches	Yes	No	Yes within permitted organization
Import CSV	Yes	Delegated only	Yes within permitted organization
Assign users and branch roles	Yes	No	Yes within permitted organization
Configure system-wide adapters	No by default	No	Yes, subject to organizational scope

An administrator does not automatically gain permission to see every restaurant chain's reviews. For each request, the backend verifies that the selected branch belongs to an organization the signed-in person is authorized to access. A manager who changes a branch identifier in the URL must still be denied if that branch is not assigned to them. The technical term *organization-level data isolation* means exactly this ownership check; it is unrelated to renting commercial premises.

6.5 Business rules and precise indicator semantics

1. **Aspect absent versus neutral.** A neutral value counts only where the aspect was actually mentioned but expressed no positive or negative orientation. An absent aspect has no `AspectResult` record.
2. **Multi-aspect review.** One review may contribute to several aspect denominators, but it remains one review in total-review counts; summing aspect counts may exceed the number of reviews.
3. **Negative share for an aspect.** `negative aspect mentions / all classified mentions of that same aspect` over the selected branch/period/model version. Report numerator and denominator. If denominator is zero, display `N/A`, not zero percent.
4. **Branch comparison.** Use equivalent time windows and show both rates and absolute review/mention counts. Explain source gaps and recent import status.
5. **Mixed languages versus mixed sentiment.** A review may switch between Russian and Kazakh while still expressing clear positive or negative sentiment; language mixing is an input property, not a fourth sentiment class. Separately, truly contradictory opinions about the *same* aspect require explicit span-splitting, adjudication, or exclusion rules.
6. **Model provenance.** Do not combine predictions from two model versions in one indicator without explicit reconciliation. A model deployment must trigger a documented reanalysis or a version-selectable query.

6.6 Traceability from requirement to artifact

Requirement cluster	Principal UI surface	Service/API responsibility	Planned proof
FR-01, FR-02	Login, route guards, scoped branch list	Spring Security, organization/branch authorization	Negative authorization integration tests
FR-03, FR-21	Map, branch directory and management	Branch CRUD, coordinates, authorized map query and list fallback	Map-pin to branch-detail E2E test, authorization and missing-coordinate tests
FR-04, FR-05, FR-06	CSV import and job history	Import endpoint, normalization, dedup, job worker	Fixture-based import/rollback/ idempotency tests
FR-07, FR-08, FR-18, NFR-01/02	RU/KK/mixed review labels and analytics	Versioned multilingual ML endpoint + analysis persistence	Held-out evaluation for each language group, API contract tests
FR-10, FR-11, FR-12	Dashboard, filters, comparisons	Scoped aggregates and paginated review API	SQL reconciliation + E2E scripted scenarios
FR-13	Export from filtered view	Authorized CSV response	Row-level data equality and UTF-8 check
NFR-04, NFR-06, NFR-08	Cross-cutting	Performance, security and usability	k6/JMeter, attack test cases, real user study

6.7 Data rights and ethical constraints

Anonymization should be considered at the research-export layer, and personal names or unique contact information should not be included merely because they appeared in publicly visible reviews. Authorized access to a business account does not automatically establish unrestricted redistribution or bulk retention rights. The final project must not promise unverified scraping. If an appropriate corpus cannot be obtained, narrow the research claims, use a licensed dataset, or substitute explicitly synthetic demo content.

7 Architecture, Data Model and System Diagrams

7.1 Logical architecture overview

ReviewLens uses four principal runtime responsibilities. **React/TypeScript** renders and requests scoped information. **Spring Boot** is the security and do-

main boundary: it authenticates users, enforces branch permissions, validates CSV imports, coordinates work and exposes analytics. **PostgreSQL** stores organization, branch, review, analysis and job entities. **Python/FastAPI** performs only internal NLP inference and returns typed results; it does not accept browser traffic or independently modify persistent business records. Permitted import adapters normalize external data into the shared schema.

The backend creates a durable import job; a worker validates, deduplicates and stores admissible reviews. The worker sends text for inference through a versioned internal contract. The backend saves the resulting label pairs and recomputes analytics. A failed optional external connector cannot block the CSV path.

7.2 Diagram 1 — System context

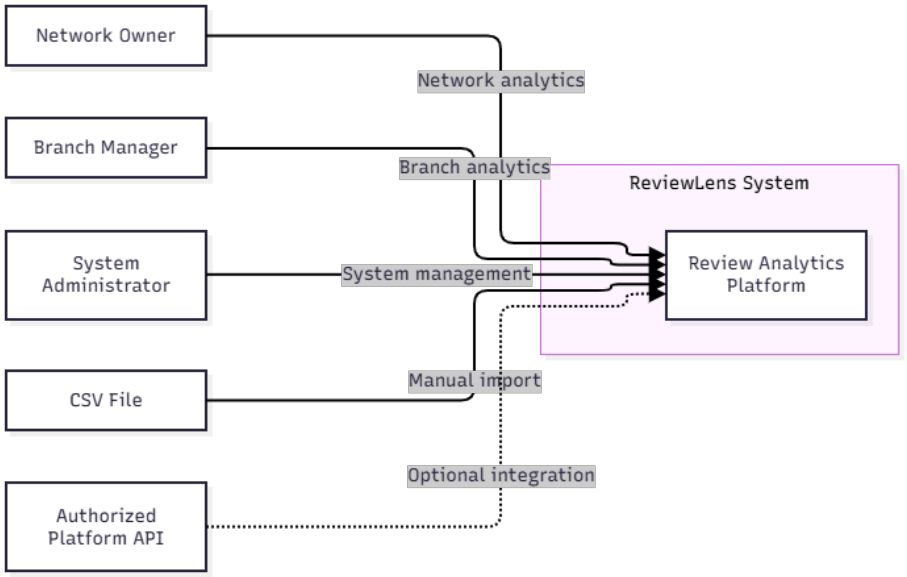


Figure 1: ReviewLens system context: owner, manager, administrator, authorized sources, and system boundary.

7.3 Diagram 2 — Use cases and RBAC actors

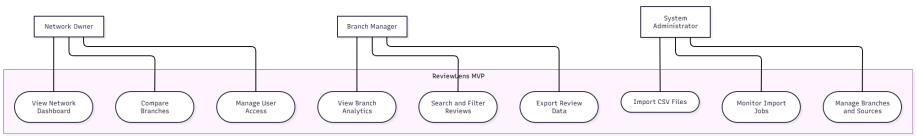


Figure 2: Use-case view of required manager, owner, and administrator workflows.

7.4 Diagram 3 — Component architecture

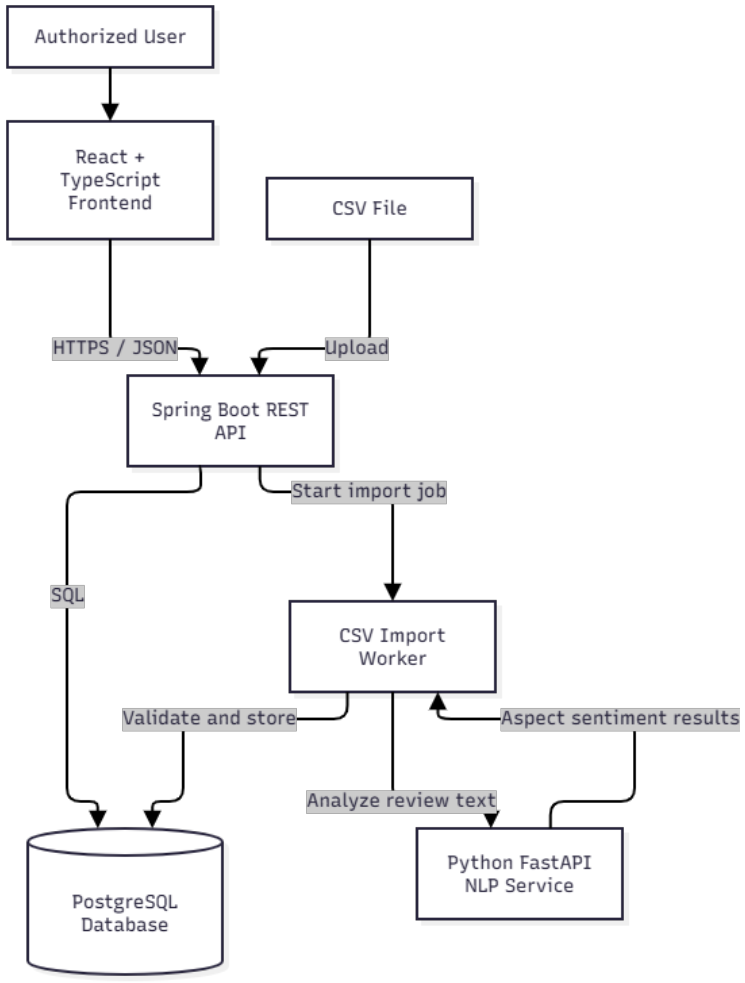


Figure 3: Component responsibilities and communication contracts.

7.5 Diagram 4 — Preliminary entity–relationship model

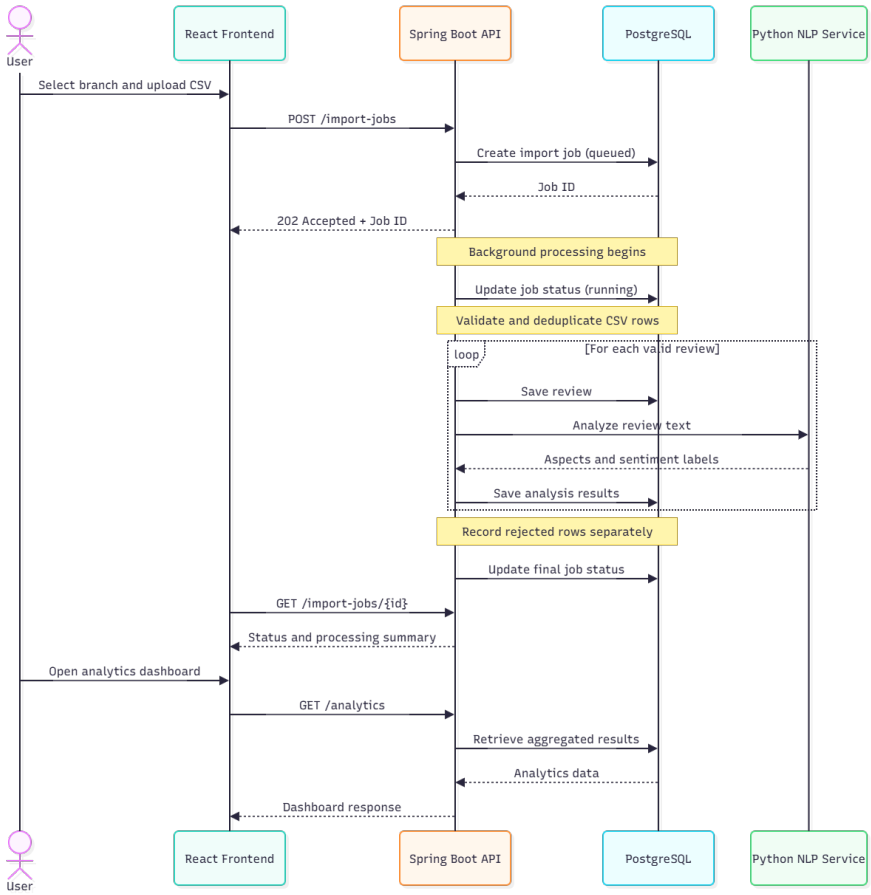


Figure 4: Preliminary normalized data model for organizations, branches, reviews, analyses and import jobs.

7.6 Diagram 5 — Import and inference sequence

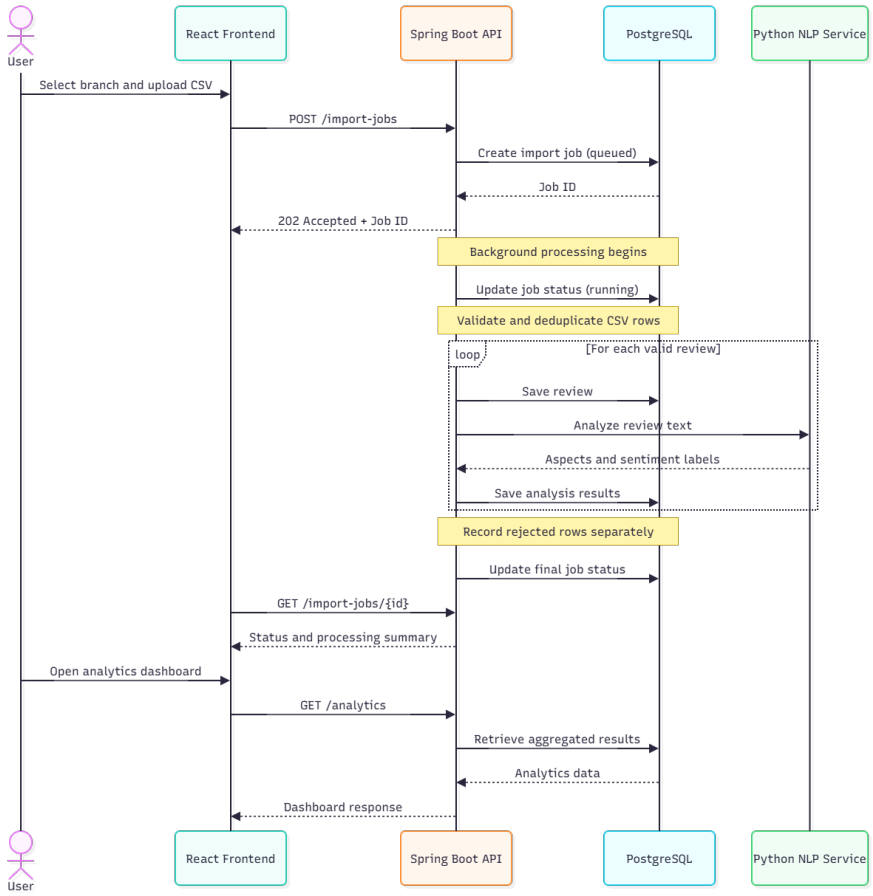


Figure 5: Authorized CSV submission, job processing, inference and dashboard refresh sequence.

7.7 Core entities and integrity constraints

Entity	Proposed key fields	Responsibility / constraints
`Organization`	`id`, `name`	Data-owner boundary; every branch belongs to exactly one organization
`User`	`id`, `organization_id`, `email`, `password_hash`, `role`	Authentication identity and organization scope

Entity	Proposed key fields	Responsibility / constraints
`UserBranch`	`user_id`, `branch_id`	Assignment of restricted managers to branches; uniqueness on pair
`Branch`	`id`, `organization_id`, `name`, `address`, `latitude`, `longitude`, `status`	Unit of reporting and map selection; coordinates optional, missing-coordinate branches remain accessible in a list
`Source`	`id`, `branch_id`, `platform`, `external_location_id`, `enabled`	Origin of each permitted record
`Review`	`id`, `source_id`, `branch_id`, `external_id`, `date`, `language`, `rating`, `text`, `content_hash`	Original multilingual text is preserved; language is ru, kk, mixed_ru_kk or unknown, with optional span-level diagnostic metadata
`Analysis`	`id`, `review_id`, `model_version`, `overall_polarity`, `processed_at`	Versioned inference on a particular review
`AspectResult`	`analysis_id`, `aspect_code`, `polarity`, `confidence`	One aspect–polarity result per aspect per analysis
`ImportJob`	`id`, `branch_id`, `status`, `total`, `accepted`, `rejected`, `errors`	Durable status and error summary

An external platform review ID permits a uniqueness constraint on `(source_id, external_id)`. If absent, the system may use a documented hash of normalized content, date and branch with caution against merging unrelated similar reviews. Every `Review.branch_id` and `Source.branch_id` should be internally consistent and covered by authorization logic. An `Analysis` should be immutable with respect to recorded model version unless an explicit correction process is documented.

7.8 API contract sketches

Public job submission — `POST /v1/import-jobs` after authorized upload and branch check:

```
{ "branch_id": "branch-101", "source_type": "csv", "upload_id":
↪  "upload-101" }
```

Proposed response `202 Accepted`:

```
{ "job_id": "job-401", "status": "queued", "status_url":
↪  "/v1/import-jobs/job-401" }
```

Internal model request — `POST /internal/v1/analyze`, accessible only to trusted backend services:

```
{
  "requested_model_version": "candidate-v1",
  "reviews": [ {
    "id": "review-001",
    "text": "Тамақ дәмді, но заказ ждали 30 минут.",
    "language_hint": "mixed_ru_kk"
  } ]
}
```

Illustrative contract response:

```
{
  "model_version": "candidate-v1",
  "results": [ {
    "id": "review-001", "language": "mixed_ru_kk", "aspects": [
      { "code": "food", "polarity": "positive", "confidence": 0.91 },
      { "code": "speed", "polarity": "negative", "confidence": 0.86 }
    ]
  } ]
}
```

A language-detection policy must define ru, kk, mixed_ru_kk and unknown, and preserve all text without machine translation silently changing a customer's meaning. An OpenAPI contract must specify errors, pagination, model version, dates, supported encodings, and numerical units before coding.

Analytics endpoints (proposal):

- GET /v1/branches — only branches visible to the signed-in user;
- GET /v1/branches/map — available coordinates and brief branch indicators for map pins;
- GET /v1/reviews, filtered by branch, date, language, aspect and polarity;
- GET /v1/analytics/aspects, filtered by branch and dates;
- GET /v1/analytics/compare, using branch IDs and the same date range.

7.9 Import and inference failure handling

The job state machine should distinguish `queued`, `running`, `completed`, `completed_with_errors`, and `failed`. One malformed CSV row must not abort otherwise valid rows unless the header/template is structurally invalid. Per-row failures should be summarized without exposing secrets or raw private text in public logs. Network timeouts permit bounded retries with an idempotency key; a persistently failed job is visible to an authorized administrator. Empty reviews are retained if policy permits but do not count as neutral language evidence and do not enter NLP inference as meaningful text.

7.10 Architecture verification checklist

Before implementation, cross-check all five diagrams against the FR/NFR list: same actor names, exactly six aspect codes, RU/KK/mixed support, authorized branch scope, map coordinates, CSV Must, optional API adapters, internal FastAPI, PostgreSQL write ownership, and the job lifecycle.

8 UX, Information Architecture and Prototype

8.1 UX objectives and content hierarchy

The interface should help a user answer: (1) *What changed?* (2) *Where did it change?* (3) *How much evidence supports the claim?* (4) *Which review examples explain the change?* Every visualization requires an explicit branch or chain scope, time range, and count of relevant reviews. Empty states, model errors, and stale imports should appear in normal human-readable text; absence of evidence should not be drawn as a healthy zero complaint rate.

8.2 Eight map-first prototype screens

Screen	Required visible information	Main operation	Failure / empty state
1. Sign-in	Account details, errors, loading	Authenticate and route to authorized workspace	Invalid password, expired session, API outage
2. Astana branch-map home	Map, permitted branch pins, short branch cards, sidebar, source freshness	Click branch pin or branch list entry	No coordinates, unassigned branches, map unavailable
3. Branch detail analytics	Selected branch, period, review n, six aspects, polarity, language groups	Examine aspect rates and open source reviews	Too few reviews; unsupported language; stale analysis
4. Chain dashboards	Same-period comparison, normalized negative aspect shares and denominators	Compare two or more own branches	Unequal coverage, sparse sample, unknown language
5. Reviews explorer	Original RU/KK/mixed text, date, source, predicted aspect labels and filters	Drill from an aspect to review evidence	No results; failed inference; model uncertainty
6. Import and sources	CSV template, branch mapping, job progress and error report	Upload, validate and retry safely	Malformed encoding, duplicates, job timeout

Screen	Required visible information	Main operation	Failure / empty state
7. Export reports	Authorized branch/date/language/aspect selection and CSV metadata	Download traceable selected report	Access denied, no matching rows, export failure
8. Users and permissions	Members, roles, assigned branches, active status	Delegate rights without exposing other organizations' data	Unauthorized route or invalid branch assignment

Home-screen navigation. Once an owner signs in, ReviewLens opens the Astana map, not a generic table. The left sidebar links to the map, dashboards, reviews, import, exports and users. Each mapped branch is a selectable point with its name and a short, carefully denominated summary. Clicking a point opens that branch's detailed analytics; the chain dashboard provides a broader comparison. Managers see only their assigned points. A textual branch list remains available when geocoding or the map service fails.

Mixed-language feedback. The reviews screen shows the untouched source text and a ru/kk/mixed label, with aspect-level positive, neutral and negative results. The language label does not imply one sentiment for the whole review, and it is possible for one sentence in mixed languages to praise food but criticize speed. Before production, the team must test erroneous language identification and whether models produce materially different errors for Kazakh-only and mixed-language reviews.

8.3 Interaction scenario U1 — Owner investigates speed complaints

1. The owner signs in and sees a map of their authorized Astana branches, then chooses a branch pin and an agreed four-week window.
2. The branch page shows the number of analyzed reviews and counts for the speed category.
3. The owner opens the dashboard from the sidebar and selects two branches. Comparison uses identical dates and presents rate *and* denominator.
4. A visual difference triggers drill-down into negative speed reviews for the higher-rate branch.
5. The owner reads original text and source/time context before drafting any operational question.
6. The application does not assert a root cause or recommend staffing changes as validated facts.

Acceptance measure: participant correctly identifies which branch has a higher *proportion of negative speed mentions*, describes the denominator, and

can find at least one supporting original review without help. The study measures task time and interpretation mistakes.

8.4 Interaction scenario U2 — Manager validates an aspect label

An assigned branch manager filters the reviews explorer to `cleanliness` + `negative`; opens a review; inspects language, original context, rating, and attached model version; and distinguishes a misclassified label from a genuine complaint. If an error-reporting UI is added, it records feedback as a review item—not an immediate retraining event. The manager cannot access other branches through query-string manipulation.

8.5 Interaction scenario U3 — Administrator imports CSV safely

An administrator downloads a documented CSV template, selects a permitted branch, uploads the file, inspects preliminary validation, and submits an import job. While processing, the interface displays accepted, duplicate, and rejected counts; after completion, the user can inspect a bounded error report and see new reviews in the list. Retrying the identical file should not double counts. If a third-party API is unavailable, the UI should offer the CSV route without an error suggesting the whole product is broken.

8.6 Prototype availability and fidelity

An editable eight-screen ReviewLens design and clickable navigation prototype is available in Figma at: ReviewLens — Astana Multilingual Prototype. It includes login, Astana map, branch detail, dashboards, reviews, CSV import, export and role management. The login button, sidebar entries and map pins navigate between designed screens.

8.7 Accessibility, responsiveness and honest analytics display

Important controls should be keyboard reachable, labels explicit, focus visible, and charts available as accessible tables when possible. Color is supplementary to positive/neutral/negative text labels. Mobile layouts may collapse branch comparison into stacked cards but must preserve date and n. The interface should display a low-sample caution, stale-source notice, unsupported-language warning, and model/analysis status. Percentages should never appear without counts where the data can be sparse.

8.8 Proposed usability tasks and screen acceptance

Task	Participant prompt	Observed data to record	Completion condition
U1	Start from the Astana map, open a branch and find which of two branches has more negative speed mentions	Pin selection, completion time, errors, branch and denominator interpretation	Correct branch and traceable original example
U2	Find negative cleanliness feedback for your branch	Filter steps, misclicks, task success	Relevant original review opened within assigned scope
U3	Import a CSV with one malformed line	Correct error interpretation, job tracking, duplicates on retry	Correct accepted/rejected counts, no duplicated review
U4	Try to open an unassigned branch	Route/action attempted, system response	Protected data inaccessible, useful denial message

9 Research Methodology, Verification and Evaluation Plan

9.1 Evaluation design: four independent questions

The evaluation has four separate tracks: **(a)** whether inputs and annotation are valid; **(b)** how well machine-learning predictions match held-out reference labels; **(c)** whether the integrated application satisfies requirements and performance/security constraints; and **(d)** whether target users can complete practical tasks using aspect visualizations. Each track needs its own sample, measurements, success criteria and limitations. There is no valid inference from one track directly to all others.

9.2 Corpus provenance, data selection and split discipline

Candidate research inputs include permitted partner-provided reviews, licensed public research resources, and explicitly synthetic text for interface tests. English SemEval restaurant corpora [S1] may inform task framing but are not gold labels for a Russian–Kazakh Kazakhstan application. If a platform does not permit collecting or retaining particular content, exclude it. Record source licensing, collection time, selection criteria, languages, duplicate policy, de-identification, and intended use. Stratify the sampling plan across **Russian-**

only, Kazakh-only and mixed-language reviews and preserve the original strings.

Before any model training, define the annotation guide and construct non-overlapping train/validation/test partitions. A starting **proposal** is 70%/15%/15%; for small classes or rare Kazakh and mixed examples, allocation may need revision before final testing. Deduplicate near-identical reviews **before** splitting, and assess grouping by source, time, or branch if multiple reviews are highly correlated. Hold out the test partition from model choice and threshold tuning. Report *counts of gold examples* for RU, KK and mixed subsets, as well as each aspect–polarity class in each subset. Do not report a group-specific F1 if its support is too low for a meaningful estimate; document the limitation and seek more valid labels. Record pseudorandom seed and corpus version hashes. For a second domain-shift evaluation, if feasible, reserve a subset of entirely unseen branches or a later time interval.

9.3 Annotation protocol and ambiguity adjudication

Each review or defined text span receives presence decisions for `food`, `service`, `price`, `atmosphere`, `cleanliness` and `speed`. For each present aspect, annotate `positive`, `neutral` or `negative`. Review-level `overall_polarity` is a separate optional field. Annotators should read both Russian and Kazakh, especially when a single sentence switches language. The guide must distinguish **staff behavior** (`service`) from **waiting or delivery time** (`speed`). A neutral factual phrase about listed menu prices is different from no mention of price. Store language identification separately from aspect and polarity labels.

Potential edge cases: short one-word reviews; emojis; sarcasm; negation; contradictory polarity for the same aspect in one record; quoted customer-service responses; Russian/Kazakh mixed language; irrelevant text; star-only entries. Never silently collapse all ambiguity into negative or positive.

Use at least two independent annotators on an overlapping subset if resources permit. Report Cohen's κ per binary aspect; assess polarity agreement where both annotators assigned the aspect; inspect confusion disagreements. If three or more annotators contribute, a justified alternative such as Fleiss' κ may be appropriate. Present the number of overlapping items, raw agreement and reconciled gold labels.

9.4 Model experiment design

Experiment	Input and model	Output to compare	Resource/cost record
E0 — frequency baseline	Always predict frequent aspects / polarity frequencies	Baseline F1; demonstrates imbalance sensitivity	Negligible compute; record label distribution
E1 — linear baseline	TF-IDF (word/character n-grams) + one-vs-rest logistic regression	Aspect Micro/Macro-F1, per-label precision/recall	CPU runtime, memory, parameters
E2 — polarity baseline	Simple lexicon/rule or independent linear classifier	Conditional polarity and end-to-end pair F1	Coverage of rules, CPU cost
E3 — multilingual transformer	Fine-tuned XLM-R [S15] or another documented encoder covering RU and KK	Held-out aspect/pair F1 split into RU-only, KK-only and mixed groups	Training duration, inference p50/p95, memory
E4 — language-specific diagnostic (optional)	Fine-tuned Russian encoder such as rubert-tiny2 on RU subset only	Whether RU-specific model changes RU-only outcomes, without extrapolation to KK	GPU/CPU requirements, size and latency

Fair comparison principles: the same test labels and evaluation scripts; comparable preprocessing; decision thresholds tuned only on validation; seeded experiments; fixed inference hardware for latency; explicit compute and configuration; no hidden hand-selection of examples. Do not cite a paper's F1 as directly comparable to the local experiment when datasets, task scopes, languages and class balance differ.

9.5 Model metrics and reporting templates

For aspect detection, report binary precision/recall/F1 for each of six categories plus Micro-F1 and Macro-F1. For aspect–polarity pair extraction, compare predicted `(aspect, polarity)` pairs against the gold set and calculate micro/macro statistics with clear inclusion rules; additionally report polarity errors conditional on correctly detected aspects. Repeat the key tables **separately for RU-only, KK-only and mixed RU–KK examples**; include the test-set count and low-support warnings for each group. Evaluate language identification with a separate ru/kk/mixed/unknown confusion matrix and inspect errors at code-switch boundaries. If FR-09 is implemented, overall positive/neutral/negative sentiment needs its own Macro-F1 and confusion matrix. A confidence interval or bootstrap variation should be reported when sample size supports it.

9.6 Error analysis and domain robustness

Manually inspect false positives and false negatives from each aspect, especially low-frequency categories such as cleanliness/atmosphere. Partition errors by negation, implicit complaints, short texts, mixed opinions, length, source, and language. A *domain shift* table can compare branches or time periods held out from training when ethically accessible data permit. Examples should be paraphrased or used only under appropriate source and privacy permissions; do not reproduce identifiable customer comments in a public appendix by default.

9.7 Software, security and reliability tests

Test group	Requirement	Planned procedure	Pass criterion or evidence
Authentication / RBAC	FR-01/02; NFR-06	Valid/invalid tokens; manager changes branch IDs; unauthorized organization/branch requests	Unauthorized access denied; no data leak
Database consistency	FR-03/06	PostgreSQL migrations, foreign-key violation, transactional rollback via Testcontainers	Constrained records; expected rollback
Import parsing	FR-04/05	Valid, empty, malformed header/row, Unicode, duplicate file, retry after failure	Consistent counts, no duplicates, bounded errors
ML contract	FR-07/08; NFR-01/02	Schema validation, unsupported language, timeout, multiple aspects, model version	Stable JSON and error codes; reproducible evaluation
Analytics	FR-10/11/12	Compare REST aggregates with reference SQL against fixture data	Exact count/denominator match; authorized scope only
Export	FR-13	Export after filters and test UTF-8 in spreadsheet reader	Correct rows, dates and role restrictions
Frontend journey	FR-10/11; NFR-10	Playwright/Vitest task flows, keyboard navigation, empty states	Predeclared screen acceptance cases pass
Load	NFR-04	k6/JMeter, 50 VUs, $\geq$10,000 reviews, 2-minute warm-up + 10-minute load	p95 analytics GET $\leq$2s target; error fraction/host specs reported

Test group	Requirement	Planned procedure	Pass criterion or evidence
Security inspection	NFR-06	IDOR, injection, XSS, secret/log checks, CSRF context, rate limits	No critical unresolved vulnerabilities in tested scope
Operational recovery	NFR-05/07	Worker interruption and idempotent replay	No silent duplicates; job state observable

Before publishing actual results, include a test environment summary: software versions, Git SHA, CPU/RAM/GPU, data size, user profile, measured percentile and sample count, percentage of failed requests, and the full script/command.

9.8 User study and managerial task comparison

Recruit actual potential users where feasible; even 3–5 interviews are exploratory rather than statistically representative. Compare two task conditions: **Condition A**, conventional star-rating and review-list exploration; **Condition B**, ReviewLens aspect dashboard and drill-down. Choose matched tasks and counterbalance the order (A→B for some participants, B→A for others) to reduce learning effects. Use the same synthetic or appropriately permitted cases, avoiding disclosure of business-sensitive text.

For each task collect elapsed time to a correct conclusion, correct/incorrect interpretation, number of source reviews opened, navigation errors, task completion, and participant comments. If enough participants are recruited, administer the System Usability Scale (SUS) with standard scoring. Report distribution and individual outcomes carefully; a tiny sample cannot establish causal improvement in business performance. A debrief should ask what information participants trusted and which indicators they found ambiguous.

9.9 Reproducible evidence record

Each experiment/test entry should include: `test_id`, date, researcher, source rights, input corpus version or fixture, Git SHA, software/hardware configuration, expected value, observed value, pass/fail, path to logs/screenshots, and limitations.

10 Project Management, Collaboration and Risks

10.1 Proposed responsibilities

Participant	Suggested primary area	Shared review obligation
Zhambulkhan Dinmukhamed Zhandosuly	React/TypeScript, UX design, corpus/annotation, ML experiments, authorized ingestion adapters	API data contract, architecture consistency, frontend/ML PR review
Aitkazyyev Serik Bekkazyuly	Spring Boot, PostgreSQL, RBAC, import jobs, domain API, integration/security tests	ML service contract, review-schema reconciliation, backend PR review
Orazova Arailym Zhanbolatkyzy	Academic guidance on scope, research question and evaluation	Supervisor feedback only after actually received

This ownership is a planning proposal derived from previous materials, not an assertion that each contributor has accepted every task.

10.2 Source control and issue management

A single project repository is recommended with `docs/`, `diagrams/`, `prototype/`, `src/frontend/`, `src/backend/`, `src/ml-service/`, and `tests/`. Architecture diagrams should keep editable `.drawio` source plus rendered PNG/SVG. Record important changes through pull requests reviewed by the second author; Git history must reflect real work and real dates. A project board (YouTrack or another approved system) should show issue owner, acceptance criteria, estimated dependency, current status and link to PR/commit.

Suggested first backlog tasks: create repository and issue board; freeze six-aspect schema; implement branch-scoped auth; establish CSV data contract; document permitted data; draft annotation protocol; implement TF-IDF baseline; define OpenAPI and ML contracts; connect six screens; add E2E and load scripts; run data/usefulness pilots; resolve supervisor notes.

10.3 Risk register

Risk	Likelihood / impact (provisional)	Warning sign	Mitigation and fallback
R01: No authorized bulk 2GIS access	High / high	No documented interface or permission	CSV remains Must; defer connector
R02: Google API not accessible to partner	Medium / high	No verified business account/OAuth approval	Demonstrate authorized CSV corpus instead
R03: Sparse labels for minority aspects	High / high	Few cleanliness/atmosphere examples	Stratified annotation, per-class reporting, honest limitations
R04: Insufficient GPU budget	Medium / medium	Training too slow or memory errors	Baseline first; smaller encoder, shorter sequences, batched CPU inference
R05: Train/test leakage	Medium / high	Duplicate reviews across split	Deduplicate before splitting; hash/group audits
R06: Cross-branch access failure	Medium / high	API accepts arbitrary `branch_id`	Server-side scope checks and negative tests
R07: Too few interview/user-test participants	High / medium	Cancellations or missing consent	Early recruitment; label outcomes exploratory
R08: Diagram/requirements/UI divergence	High / high	Features appear in screen but not FR	Traceability matrix and cross-author review
R09: Source policy/API changes	Medium / high	Adapter failure or changed fields	Adapter isolation and CSV fallback
R10: Supervisor feedback delayed	Medium / high	No response before scope decisions	Submit materials early; log pending decisions, never infer approval
R11: False managerial inference from sparse data	Medium / high	Large percentages with n of 1–2	Show n, uncertainty, source freshness, drill-down
R12: Undocumented personal-data retention	Medium / high	Raw identifying fields in logs/repo	Data minimization, access controls and retention review
R13: Insufficient Kazakh and code-switched training evidence	High / high	Very few independently annotated KK/mixed reviews or weak group-specific F1	Recruit bilingual annotators, expand lawful samples, display low-confidence warnings and report limitations without claiming parity

10.4 Definition of done

A feature is complete only when its acceptance criteria pass, relevant documentation and diagrams are updated, permission/error paths have been tested, code

has been reviewed, and a reproducible evidence link exists. An academic experiment is complete only with fixed data/split versions, comparable baselines, archived results, and honest discussion of negative findings.

11 Academic Bibliography

The bibliography below retains the original fourteen works and adds the multilingual encoder reference [S15] used in the revised literature matrix.

[**S1**] Pontiki, M., Galanis, D., Papageorgiou, H., Androutsopoulos, I., Manandhar, S., AL-Smadi, M., Al-Ayyoub, M., Zhao, Y., Qin, B., De Clercq, O., Hoste, V., Apidianaki, M., Tannier, X., Loukachevitch, N., Kotelnikov, E., Bel, N., Jiménez-Zafra, S. M., & Eryiğit, G. (2016). SemEval-2016 Task 5: Aspect based sentiment analysis. In *Proceedings of the 10th International Workshop on Semantic Evaluation (SemEval-2016)* (pp. 19–30). Association for Computational Linguistics. https://doi.org/10.18653/v1/S16-1002

[**S2**] Hua, Y. C., Denny, P., Wicker, J., & Taskova, K. (2024). A systematic review of aspect-based sentiment analysis: Domains, methods, and trends. *Artificial Intelligence Review, 57*, Article 296. https://doi.org/10.1007/s10462-024-10906-z

[**S3**] Rogers, A., Romanov, A., Rumshisky, A., Volkova, S., Gronas, M., & Gribov, A. (2018). RuSentiment: An enriched sentiment analysis dataset for social media in Russian. In *Proceedings of the 27th International Conference on Computational Linguistics* (pp. 755–763). Association for Computational Linguistics. https://aclanthology.org/C18-1064/

[**S4**] Kotelnikova, A., Paschenko, D., Bochenina, K., & Kotelnikov, E. (2022). Lexicon-based methods vs. BERT for text sentiment analysis. In *Analysis of Images, Social Networks and Texts* (Lecture Notes in Computer Science, Vol. 13217, pp. 71–83). Springer. https://doi.org/10.1007/978-3-031-16500-9_7

[**S5**] Smetanin, S., & Komarov, M. (2019). Sentiment analysis of product reviews in Russian using convolutional neural networks. In *2019 IEEE 21st Conference on Business Informatics* (pp. 482–486). IEEE. https://doi.org/10.1109/CBI.2019.00062

[**S6**] Zmitrovich, D., Abramov, A., Kalmykov, A., Kadulin, V., Tikhonova, M., Taktasheva, E., Astafurov, D., Baushenko, M., Snegirev, A., Shavrina, T., Markov, S. S., Mikhailov, V., & Fenogenova, A. (2024). A family of pretrained transformer language models for Russian. In *Proceedings of LREC-COLING 2024* (pp. 507–524). ELRA and ICCL. https://aclanthology.org/2024.lrec-main.45/

[**S7**] cointegrated. (n.d.). *rubert-tiny2* [Model card]. Hugging Face. https://huggingface.co/cointegrated/\protect\penalty\z@rubert-t

iny2

[**S8**] Yeshpanov, R., & Varol, H. A. (2024). KazSAnDRA: Kazakh sentiment analysis dataset of reviews and attitudes. In *Proceedings of LREC-COLING 2024* (pp. 9657–9667). ELRA and ICCL. https://aclanthology.org/2024.lrec-main.844/

[**S9**] Grootendorst, M. (2022). *BERTopic: Neural topic modeling with a class-based TF-IDF procedure* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2203.05794

[**S10**] Jayakody, D., Thenahandi, P., & Jayarathna, S. (2026). *SentimentLens: Reconciling sentiment and ratings via dual-modality in the hospitality sector* [Preprint]. arXiv. https://doi.org/10.48550/arXiv.2606.00084

[**S11**] Scaria, K., Gupta, H., Goyal, S., Sawant, S., Mishra, S., & Baral, C. (2024). InstructABSA: Instruction learning for aspect based sentiment analysis. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)* (pp. 720–736). Association for Computational Linguistics. https://doi.org/10.18653/v1/2024.naacl-short.63

[**S12**] Fehle, J., Münster, L., Schmidt, T., & Wolff, C. (2023). Aspect-based sentiment analysis as a multi-label classification task on the domain of German hotel reviews. In *Proceedings of KONVENS 2023* (pp. 202–218). https://aclanthology.org/2023.konvens-main.21/

[**S13**] Rybakov, V., & Malafeev, A. (2018). Aspect-based sentiment analysis of Russian hotel reviews. In *Supplementary Proceedings of the 7th International Conference on Analysis of Images, Social Networks and Texts* (pp. 75–84). CEUR Workshop Proceedings. https://ceur-ws.org/Vol-2268/paper8.pdf

[**S14**] Loukachevitch, N., Blinov, P., Kotelnikov, E., Rubtsova, Y., Ivanov, V., & Tutubalina, E. (2015). SentiRuEval: Testing object-oriented sentiment analysis systems in Russian. *Computational Linguistics and Intellectual Technologies, 2*(14), 3–15. https://dialogue-conf.org/digests/dialog2015/materials/pdf/LoukachevitchNVetal.pdf

[**S15**] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L., & Stoyanov, V. (2020). Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics* (pp. 8440–8451). Association for Computational Linguistics. https://aclanthology.org/2020.acl-main.747/

12 Official Product and Technology Source Register

[**W1**] **Google Business Profile.** "Manage customer reviews": https://support.google.com/business/answer/3474050?hl=en ; "Work with review data — Business Profile APIs": https://developers.google.com/my-business/content/review-data ; multi-location management: https://support.google.com/business/answer/3370250 .

[**W2**] **2GIS Business Help.** Customer review workflow: https://help.2gis.ru/question/kak-rabotat-s-otzyvami-klientov-instrukciya-dlya-kompaniy ; ratings: https://help.2gis.ru/question/otkuda-beryotsya-reyting-kompaniy .

[**W3**] **Yandex Business.** Review management: https://yandex.com/support/business-priority/en/manage/reviews .

[**W4**] **ReviewTrackers.** Experience analytics: https://www.reviewtrackers.com/blog/reviewtrackers-experience-analytics/ ; performance by location: https://www.reviewtrackers.com/blog/performance-score/ .

[**W5**] **Birdeye.** Multi-location review reports: https://birdeye.com/review-reports/ ; Insights AI guide: https://support.birdeye.com/en/articles/12654829-getting-started-with-insights-ai .

Technology primary documentation (to lock versions): React https://react.dev/ ; Spring Boot https://spring.io/projects/spring-boot ; PostgreSQL https://www.postgresql.org/docs/ ; FastAPI https://fastapi.tiangolo.com/ ; XLM-R model card https://huggingface.co/FacebookAI/xlm-roberta-base ; Russian-only diagnostic model card [S7].

A Proposed Minimal CSV Schema

CSV field	Required?	Example (synthetic)	Validation rule
branch_external_id	Yes	BR-001	Maps to authorized internal branch
source_platform	Yes	partner_export	Defined platform/source code
review_external_id	Preferred	r-901	Stable dedup identifier if supplied
review_date	Yes	2026-10-01	ISO date, timezone policy documented

CSV field	Required?	Example (synthetic)	Validation rule
`text`	Conditional	*Original RU/KK review text*	UTF-8, preserve original script and code-switches; empty text is not inferred
`rating`	Optional	`4`	Source-defined scale recorded; never assume without normalization
`language`	Optional	`mixed_ru_kk`	Validate ru, kk, mixed_ru_kk, or unknown; user hint may require verification

The import should reject mismatched organization/branch ownership and refuse unsafe formula injection in spreadsheet exports. CSV columns and date policy require implementation-level agreement.

B Sample Annotation Cases for Reviewer Discussion

Synthetic review sentence	Aspect annotation	Important rule
Тамақ дәмді, но заказ ждали долго.	`food: positive`; `speed: negative`	Code-switching within a review; aspect-specific opposite polarity
Тағам өте дәмді, қызмет жақсы.	`food: positive`; `service: positive`	Kazakh-only source; a Russian-only model is not sufficient
Еда вкусная, но официант грубый.	`food: positive`; `service: negative`	Russian-only example and two independent aspects
"The restaurant is on Central Avenue."	No listed aspect, unless taxonomy explicitly considers a relevant opinion	Factual location is not sentiment-neutral food feedback
"The staff were polite and the table was clean."	`service: positive`; `cleanliness: positive`	Distinct operational concepts
"The prices are listed on the menu."	`price: neutral` only if this is accepted as an explicit aspect mention	Presence and sentiment are different tasks